



# **MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING**

*(A Unit of Rajalaxmi Education Trust®, Mangalore)  
Autonomous Institute affiliated to VTU, Belagavi, Approved by AICTE, New Delhi Accredited by  
NAAC with A+ Grade & ISO 9001:2015 Certified Institution*

## **DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

### **Event-Ease : Event Registration System (MERN Stack)**

#### **Submitted by**

**4MT23CS023**

**Ananya Hebbar**

**4MT23CS025**

**Anjali P**

**Subject: Full Stack Development**

**Course Code: 23CSPE311**

**Faculty : Dr. Clara Kanmani A**

**Semester: 05**

**Academic Year: 2025-26**

# Event-Ease :Online Event Management & Registration

## System Report

### **1. Introduction:**

Event-Ease is a web-based event registration system developed as part of the mini-project requirement. The primary aim of this project is to create a simple yet efficient platform where users can register for different events such as Workshops, Seminars, and Conferences. The system is built using the MERN stack—MongoDB, Express, React, and Node.js—which enables smooth interaction between the user interface, backend server, and database.

This application demonstrates how modern web technologies can be used to collect, store, manage, and display user information in real time. The frontend offers an intuitive form through which users enter their details, and this data is then securely stored in MongoDB Atlas using a REST API created with Node.js and Express. The stored data is automatically displayed in the interface in the form of a table, supporting additional functionalities such as search, edit, delete, and CSV download.

Overall, this project strengthens the understanding of full-stack development concepts, API communication, and database management while creating a practical application suitable for real-world use.

### **2. Technologies Used:**

This project incorporates several modern web development technologies, each contributing to the smooth functioning of the Event-Ease registration system. The combination of frontend, backend, database, and development tools ensures a complete and efficient MERN stack application.

#### **(a)Frontend Technologies:**

- **React JS:** Used to build the user interface and manage state.
- **Vite:** A fast development environment for React applications.
- **JavaScript (ES6):** Provides dynamic behaviour to the frontend.
- **HTML5 & CSS3:** Used for page structure and styling.
- **Fetch API:** Used to communicate with the backend server.

#### **(b)Backend Technologies:**

- **Node.js:** Powers the backend logic and server execution.
- **Express.js:** Creates RESTful API endpoints for communication.
- **Mongoose:** Acts as an ODM for interacting with MongoDB.

### Database

- **MongoDB Atlas:** A cloud-based NoSQL database used to store all event registrations.

### Tools

- **VS Code:** Code editor used for development.
- **MongoDB Compass:** Used to visually inspect stored data.
- **Browser Developer Tools:** For debugging and API testing.

## 3. Important Codes used:

### ❖ FOR BACKEND:

#### 3.1 Backend – server.js

```
eventease-backend > JS server.js > ...
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const cors = require("cors");
4
5  const app = express();
6
7  app.use(express.json());
8  app.use(cors());
9
10 // ROUTES
11 const eventRoutes = require("../routes/eventRoutes");
12 app.use("/api/events", eventRoutes);
13
14 // MONGO CONNECTION (UPDATED PASSWORD ENCODED)
15 const mongoURI = "mongodb+srv://Anu_06:Nanna%402006@eventeasecluster.fvq1zsa.mongodb.net/EventEase?retryWrites=true&w=majority";
16
17
18
19 mongoose.connect(mongoURI)
20   .then(() => console.log("✅ MongoDB Connected Successfully!"))
21   .catch((err) => console.log("❌ DB Error:", err));
22
23 // START SERVER
24 app.listen(5000, () => {
25   console.log("🚀 Server running on port 5000");
26 });
27
```

This file is the heart of the backend. It initializes the Express server, connects to MongoDB Atlas using a secure connection string, and activates the API routes. The server listens on port **5000**, and upon successful database connection, it prints a confirmation message. This ensures that both the backend server and database are functioning normally before handling any user requests.

## 3.2 Backend – eventRoutes.js

```
eventease-backend > routes > JS eventRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const Event = require("../models/Event");
4
5  // GET ALL EVENTS
6  router.get("/", async (req, res) => {
7    try {
8      const events = await Event.find();
9      res.json(events);
10   } catch (err) {
11     res.status(500).json({ error: err.message });
12   }
13 });
14
15 // POST (Add new event)
16 router.post("/", async (req, res) => {
17   try {
18     console.log("Incoming registration data:", req.body);
19     const newEvent = new Event(req.body);
20     await newEvent.save();
21     res.json(newEvent);
22   } catch (err) {
23     res.status(500).json({ error: err.message });
24   }
25 });
26
27 module.exports = router;
28
```

This file contains the API routes for performing CRUD operations.

- The **GET route** fetches all registered participants from the database.
- The **POST route** receives new registration data from the frontend, creates a new document, and saves it to MongoDB.
- The **DELETE route** removes a specific record based on its ID.

These routes form the bridge between the frontend and MongoDB, enabling real-time data transfer.

## 3.3 Backend – Event.js (Schema Model)

```
eventease-backend > models > JS Event.js > ...
1  const mongoose = require("mongoose");
2
3  const eventSchema = new mongoose.Schema({
4    name: { type: String, required: true },
5    email: { type: String, required: true },
6    phone: { type: String, required: true },
7    event: { type: String, required: true },
8  }, { timestamps: true });
9
10 module.exports = mongoose.model("Event", eventSchema);
```

This file defines the structure of each event registration using a Mongoose schema. Each document contains a name, email, phone, and event type. Mongoose automatically adds createdAt and updatedAt timestamps. This schema ensures consistent data storage across all entries.

## ❖ FOR FRONTEND:

### 3.4 Frontend – RegistrationForm.jsx

```
eventease-frontend > src > components > ⚙ RegistrationForm.jsx > ...
1  import React, { useState, useEffect } from "react";
2  import RegistrationTable from "../RegistrationTable";
3  import Toast from "../Toast";
4
5  const API_URL = "http://localhost:5000/api/events";
6
7  const RegistrationForm = () => {
8    const [name, setName] = useState("");
9    const [email, setEmail] = useState("");
10   const [phone, setPhone] = useState("");
11   const [eventType, setEventType] = useState("");
12   const [data, setData] = useState([]);
13   const [toast, setToast] = useState("");
14
15   // Load all registrations
16   const fetchRegistrations = async () => {
17     try {
18       const res = await fetch(API_URL); // ← FIXED (no /all)
19       const json = await res.json();
20       setData(json);
21     } catch (err) {
22       console.error("Error fetching registrations. Check backend!", err);
23     }
24   };
25
26   useEffect(() => {
27     fetchRegistrations();
28   }, []);
29
30   // Register new user
31   const handleSubmit = async (e) => {
32     e.preventDefault();
33     if (!name || !email || !phone || !eventType)
34       return alert("Please fill all fields!");
35
36     const newReg = { name, email, phone, event: eventType };
37
38     try {
39       const res = await fetch(API_URL, { // ← FIXED (no /register)
40         method: "POST",
41         headers: { "Content-Type": "application/json" },
42         body: JSON.stringify(newReg)
43       });
44
45       if (res.ok) {
```

```

    if (res.ok) {
      setToast("✅ Registration Successful!");
      setName("");
      setEmail("");
      setPhone("");
      setEventType("");
      fetchRegistrations(); // refresh table
    } else {
      alert("Registration failed!");
    }
  } catch (err) {
    console.error("Submit Error:", err);
    alert("Server error! Check backend.");
  }
};

// Delete
const deleteData = async (id) => {
  try {
    await fetch(`${API_URL}/${id}`, { method: "DELETE" });
    fetchRegistrations();
  } catch (err) {
    console.error("Delete Error:", err);
  }
};

// Edit
const editData = async (reg) => {
  setName(reg.name);
  setEmail(reg.email);
  setPhone(reg.phone);
  setEventType(reg.event);
  await deleteData(reg._id);
  window.scrollTo({ top: 0, behavior: "smooth" });
};

// Download CSV
const downloadCSV = () => {
  if (!data.length) return alert("No data!");
  const csv =
    "Name,Email,Phone,Event\n" +
    data.map(r => `${r.name},${r.email},${r.phone},${r.event}`).join("\n");

```

```

    const blob = new Blob([csv], { type: "text/csv" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "participants.csv";
    a.click();
  };

  return (
    <div className="container">
      <h2>Event Registration Form</h2>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Full Name"
          value={name}
          onChange={(e)=>setName(e.target.value)}
          required
        />
        <input
          type="email"
          placeholder="Email Address"
          value={email}
          onChange={(e)=>setEmail(e.target.value)}
          required
        />
        <input
          type="tel"
          placeholder="Phone Number"
          value={phone}
          onChange={(e)=>setPhone(e.target.value)}
          required
        />
        <select
          value={eventType}
          onChange={(e)=>setEventType(e.target.value)}
          required
        >
          <option value="">Select Event Type</option>
          <option value="Workshop">Workshop 🏠</option>

```



```

      <option value="Workshop">Workshop </option>
      <option value="Seminar">Seminar </option>
      <option value="Conference">Conference </option>
    </select>

    <button type="submit">Register</button>
  </form>

  <RegistrationTable
    data={data}
    editData={editData}
    deleteData={deleteData}
  />

  <div className="actions">
    <button onClick={downloadCSV}>Download CSV</button>
    <button onClick={() => window.print()}>Print</button>
  </div>

  {toast && <Toast message={toast} setToast={setToast} />}
</div>
);
};

export default RegistrationForm;

```

This component handles all user interactions, including form input, submitting registration data, fetching stored records, and displaying toast notifications. The `fetchRegistrations()` function loads all entries from the backend, while the `handleSubmit()` function sends new data to the server. This component is responsible for the core functionality of the frontend.

### 3.5 Frontend – RegistrationTable.jsx

```

eventease-frontend > src > components > RegistrationTable.jsx > ...
1  import React, { useState } from "react";
2
3  const RegistrationTable = ({ data, editData, deleteData }) => {
4    const [search, setSearch] = useState("");
5
6    const filtered = data.filter(
7      r => r.name.toLowerCase().includes(search.toLowerCase()) ||
8          r.email.toLowerCase().includes(search.toLowerCase())
9    );
10
11    let w=0, s=0, c=0;
12    filtered.forEach(r => { if(r.event==="Workshop") w++; else if(r.event==="Seminar") s++; else c++; });
13
14    return (
15      <>
16        <input type="text" placeholder="Search..." value={search} onChange={e => setSearch(e.target.value)} />
17        <table>
18          <thead>
19            <tr><th>Name</th><th>Email</th><th>Phone</th><th>Event</th><th>Actions</th></tr>
20          </thead>
21          <tbody>
22            {filtered.map(r => (
23              <tr key={r._id}>
24                <td>{r.name}</td>
25                <td>{r.email}</td>
26                <td>{r.phone}</td>
27                <td>{r.event}</td>
28                <td>
29                  <button onClick={() => editData(r)}>Edit</button>
30                  <button onClick={() => deleteData(r._id)}>Delete</button>
31                </td>
32              </tr>
33            ))}
34          </tbody>
35        </table>
36        <div>Total: {filtered.length} | Workshop: {w} | Seminar: {s} | Conference: {c}</div>
37      </>
38    );
39  };
40
41  export default RegistrationTable;

```

This file displays all user registrations in a table. It includes advanced features such as:

- Searching by name or email
- Editing an existing record
- Deleting a record
- Showing counts for Workshops, Seminars, and Conferences

This improves usability and gives the admin full control over the stored data.

### 3.6 Frontend – App.jsx

```
eventease-frontend > src > App.jsx > ...
1  import React from "react";
2  import Hero from "../components/Hero";
3  import RegistrationForm from "../components/RegistrationForm";
4  import "../App.css";
5
6  function App() {
7    return (
8      <>
9        <Hero />
10       <RegistrationForm />
11       <footer>
12         |  Contact: eventease@gmail.com |  EventEase © 2025
13       </footer>
14     </>
15   );
16 }
17
18 export default App;
```

This is the root file that combines the Hero banner and Registration form. It also includes a website footer. It represents the structural layout of the website.

## 4. Website and Explanation with outputs:

### ❖ Overall Workflow

The Event-Ease application follows a clear and structured **MERN Stack architecture**, where each layer has a specific responsibility. The system ensures smooth interaction between the user interface, backend API, and database through a sequence of well-defined operations.

### 1. Client Layer – React Frontend

The frontend is built using React, where users interact with the event registration form. It performs the following functions:



- Collects user inputs such as **name, email, phone number, and event type**.
- Sends this data to the backend via **HTTP requests** using the Fetch API.
- Fetches all registrations from the backend and dynamically displays them in a table.
- Allows users to **search, edit, delete, and download CSV** for all stored records.
- Renders instant UI feedback such as toast messages upon successful registration.

## 2. Server Layer – Node.js + Express

The backend server acts as the communication bridge between frontend and database. It performs the following:

- Receives incoming API requests (GET, POST, DELETE).
- Validates and processes registration data.
- Interacts with MongoDB through Mongoose to store or retrieve documents.
- Sends JSON responses back to the frontend for UI updates.
- Ensures secure and reliable connection to MongoDB Atlas.

## 3. Database Layer – MongoDB Atlas

The cloud-hosted MongoDB database stores all participant registrations. It is responsible for:

- Holding data permanently as documents.
- Ensuring high availability and quick access.
- Providing automatic timestamps
- Allowing queries using Mongoose ODM.

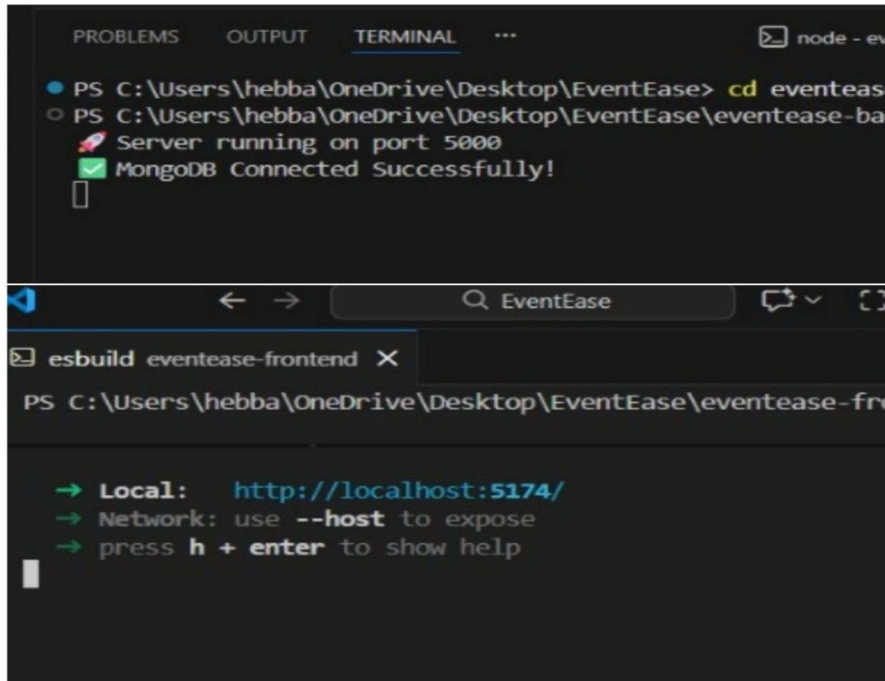
```
React (Frontend) → Express API (Backend) → MongoDB Atlas (Database)
React (UI) ← Express API Response ← MongoDB Stored Data
```

## OUTPUTS:

### 1. Figure 1– Backend and Frontend Running Successfully

This screenshot demonstrates the successful initialization of both the frontend and backend servers. The backend terminal shows the message “**MongoDB Connected Successfully!**”, confirming that the connection to MongoDB Atlas has been properly established. The server also prints “**Server running on port 5000**”, indicating that the Express API is active and ready to handle requests.

On other hand, the React application is launched using **npm run dev**, opening at <http://localhost:5174/>. This dual-server setup verifies that the environment is correctly configured, and the application is fully prepared for registering and managing user data.



## **2. Figure 2 – Homepage Display with Event-Ease Banner**

This screenshot captures the home page layout of Event-Ease. It features a visually appealing **hero section** with a blurred background image and a welcoming title, **“Welcome to Event-Ease”**, followed by a brief description of the system’s purpose. A prominent **“Register Now”** button guides the user smoothly to the registration form. This introduction creates a professional first impression and improves user engagement.

The screenshot shows a web browser window with the URL 'localhost:5174'. The page features a header with the text 'Welcome to EventEase' and a sub-header 'Register for exciting Workshops, Seminars & Conferences today!'. A 'Register Now' button is visible. Below the header is a form titled 'Event Registration Form' with three input fields: 'Full Name' (containing 'Saranya'), 'Email Address' (containing 'saru@gmail.com'), and 'Phone Number' (containing '6668885554').

### 3. Figure 3 – Registration Form (Before Submission)

In this screenshot, the registration form is displayed before user submission. It contains input fields for **Full Name**, **Email Address**, **Phone Number**, and an event type dropdown offering **Workshop**, **Seminar**, and **Conference** options. All fields are mandatory, ensuring data integrity. The user-friendly design helps reduce mistakes and enhances input accuracy. This section forms the core of user interaction in the system.

The screenshot shows the same 'Event Registration Form' after submission. The form fields are now populated with the following data:

| Name   | Email                     | Phone       | Event      | Actions                                     |
|--------|---------------------------|-------------|------------|---------------------------------------------|
| anu    | anu@gmail.com             | 09845730406 | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |
| Jyothi | anujyothi198106@gmail.com | 09845730406 | Conference | <a href="#">Edit</a> <a href="#">Delete</a> |
| Vamshi | vamsi@gmail.com           | 7204674330  | Seminar    | <a href="#">Edit</a> <a href="#">Delete</a> |
| Raghu  | raghu@gamilo.com          | 9902244880  | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |

## 4. Figure 4 – Registration Successful with Toast Message

Once a user submits the form, the data is immediately sent to the backend API through a POST request. When the backend successfully stores the information in MongoDB Atlas, the frontend displays a **green toast message saying “Registration Successful!”**. The newly added record appears instantly in the table below without needing a page refresh. This live update confirms the effectiveness of API communication and efficient React state management.

Register

Search...

| Name        | Email                     | Phone       | Event      | Actions                                       |
|-------------|---------------------------|-------------|------------|-----------------------------------------------|
| anu         | anu@gmail.com             | 09845730406 | Workshop   | <button>Edit</button> <button>Delete</button> |
| Jyothi      | anujyothi198106@gmail.com | 09845730406 | Conference | <button>Edit</button> <button>Delete</button> |
| Vamshi      | vamsi@gmail.com           | 7204674330  | Seminar    | <button>Edit</button> <button>Delete</button> |
| Raghu       | raghu@gamilo.com          | 9902244880  | Workshop   | <button>Edit</button> <button>Delete</button> |
| Geethanjali | gee@gmail.com             | 1234567890  | Seminar    | <button>Edit</button> <button>Delete</button> |
| Ananya      | akuh@gmail.com            | 7894561235  | Workshop   | <button>Edit</button> <button>Delete</button> |
| Abc         | abc@gmail.com             | 1234567892  | Seminar    | <button>Edit</button> <button>Delete</button> |
| Krishnaveni | kris@gmail.com            | 9874563210  | Conference | <button>Edit</button> <button>Delete</button> |
| Akanksha    | akki@gmail.com            | 8088456973  | Conference | <button>Edit</button> <button>Delete</button> |
| Saranya     | saru@gmail.com            | 6668885554  | Seminar    | <button>Edit</button> <button>Delete</button> |

Total: 10 | Workshop: 3 | Seminar: 4 | Conference: 3

Download CSV

Print

✓ Registration Successful!

## 5. Participants Table Display (With Filtering & Actions)

This screenshot shows the dynamic participants table containing all registered users. The table includes columns for **Name**, **Email**, **Phone**, **Event**, and **Actions**.

It demonstrates several key features:

✓ **Search Functionality**

- The user can type into the search bar, and the table instantly filters the results.
- Filtering is applied on both **name and email** fields.
- The screenshot example shows filtering results for “anu”.

✓ **Edit Function**

- Clicking **Edit** fills the form with existing details.
- The original entry is deleted and replaced upon re-submission.

✓ **Delete Function**

- Clicking **Delete** removes the selected registration from the database.
- The table updates automatically in real time.

✓ **Event Count Summary**

At the bottom, the system displays:

- Total registrations
- Count of Workshop participants
- Count of Seminar participants
- Count of Conference participants

✓ **Download CSV**

Exports all participant records into a CSV file with a single click. This feature is useful for maintaining offline records or integrating with spreadsheet-based reporting tools.

✓ **Print**

Opens the browser’s print preview, allowing the entire participant list to be printed or saved as a PDF.

Overall, this interface view demonstrates how the system effectively supports data visualization, record management, and reporting functions, making Event-Ease a complete and user-friendly event registration platform.

**Event Registration Form**

Full Name  Please fill out this field.

Email Address

Phone Number

Select Event Type

**Register**

| Name   | Email                     | Phone       | Event      | Actions                                                                   |
|--------|---------------------------|-------------|------------|---------------------------------------------------------------------------|
| anu    | anu@gmail.com             | 09845730406 | Workshop   | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |
| Jyothi | anujyothi198106@gmail.com | 09845730406 | Conference | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |

Total: 2 | Workshop: 1 | Seminar: 0 | Conference: 1

Register Now

### Event Registration Form

Full Name

Email Address

Phone Number

Select Event Type

Register

Search...

| Name        | Email                     | Phone       | Event      | Actions                                     |
|-------------|---------------------------|-------------|------------|---------------------------------------------|
| anu         | anu@gmail.com             | 09845730406 | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |
| Jyothi      | anujyothi198106@gmail.com | 09845730406 | Conference | <a href="#">Edit</a> <a href="#">Delete</a> |
| Vamshi      | vamsi@gmail.com           | 7204674330  | Seminar    | <a href="#">Edit</a> <a href="#">Delete</a> |
| Raghu       | raghu@gamilo.com          | 9902244880  | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |
| Geethanjali | gee@gmail.com             | 1234567890  | Seminar    | <a href="#">Edit</a> <a href="#">Delete</a> |
| Ananya      | aku@gmail.com             | 7894561235  | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |
| Abc         | abc@gmail.com             | 1234567892  | Seminar    | <a href="#">Edit</a> <a href="#">Delete</a> |
| Krishnaveni | kris@gmail.com            | 9874563210  | Conference | <a href="#">Edit</a> <a href="#">Delete</a> |

Print 3 pages

Destination Save as PDF

Pages All

Layout Landscape

More settings

Save Cancel

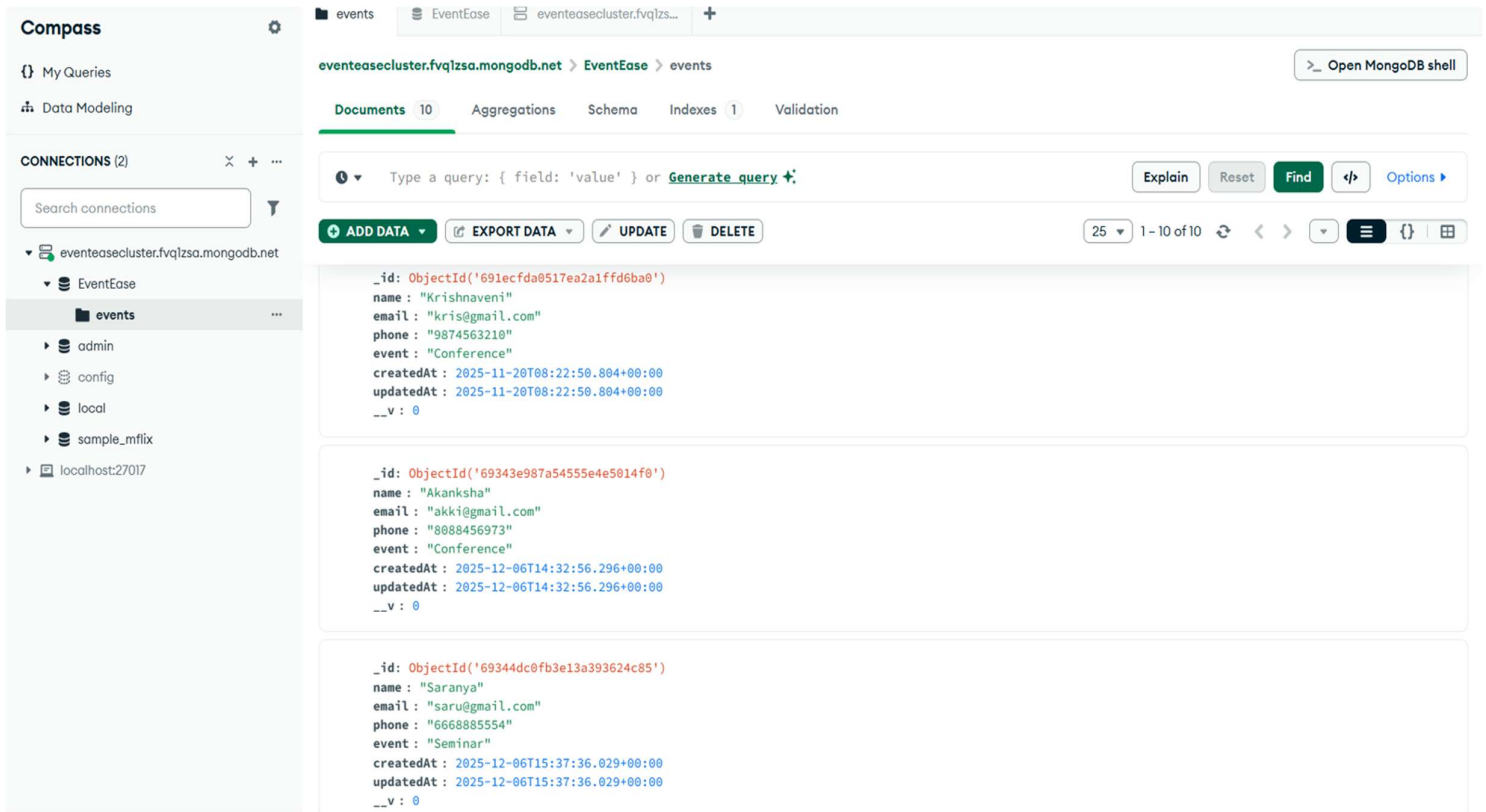
| Name   | Email                     | Phone       | Event      | Actions                                     |
|--------|---------------------------|-------------|------------|---------------------------------------------|
| anu    | anu@gmail.com             | 09845730406 | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |
| Jyothi | anujyothi198106@gmail.com | 09845730406 | Conference | <a href="#">Edit</a> <a href="#">Delete</a> |
| Vamshi | vamsi@gmail.com           | 7204674330  | Seminar    | <a href="#">Edit</a> <a href="#">Delete</a> |
| Raghu  | raghu@gamilo.com          | 9902244880  | Workshop   | <a href="#">Edit</a> <a href="#">Delete</a> |

participants (3).csv  
504 B • Done

## 6. Figure 6 – MongoDB Compass Showing Stored Documents

This screenshot confirms that all records submitted through the frontend are successfully stored in the **MongoDB Atlas database**. Each document includes fields for **name**, **email**, **phone**, **event**, along with **auto-generated timestamps**. MongoDB Compass provides a clean visual representation of this data, enabling easy verification and debugging. This screenshot validates backend-to-database communication.





## 7. Figure 7 – API Output (http://localhost:5000/api/events)

This screenshot shows raw JSON output of all stored registrations. When accessed through the browser, this endpoint retrieves all documents from MongoDB. This verifies that the backend's GET route is functioning correctly and can handle frontend requests efficiently.



```
localhost:5000/api/events
Pretty-print ☒
{
  "event": "Workshop",
  "createdAt": "2025-11-20T07:17:29.324Z",
  "updatedAt": "2025-11-20T07:17:29.324Z",
  "__v": 0
},
{
  "_id": "691eca0f8c6ca2339c73b31c",
  "name": "Abc",
  "email": "abc@gmail.com",
  "phone": "1234567892",
  "event": "Seminar",
  "createdAt": "2025-11-20T07:58:07.965Z",
  "updatedAt": "2025-11-20T07:58:07.965Z",
  "__v": 0
},
{
  "_id": "691ecfda0517ea2a1ffd6ba0",
  "name": "Krishnaveni",
  "email": "kris@gmail.com",
  "phone": "9874563210",
  "event": "Conference",
  "createdAt": "2025-11-20T08:22:50.804Z",
  "updatedAt": "2025-11-20T08:22:50.804Z",
  "__v": 0
},
{
  "_id": "69343e987a54555e4e5014f0",
  "name": "Akanksha",
  "email": "akki@gmail.com",
  "phone": "8088456973",
  "event": "Conference",
  "createdAt": "2025-12-06T14:32:56.296Z",
  "updatedAt": "2025-12-06T14:32:56.296Z",
  "__v": 0
},
{
  "_id": "69344dc0fb3e13a393624c85",
  "name": "Saranya",
  "email": "saru@gmail.com",
  "phone": "6668885554",
  "event": "Seminar",
  "createdAt": "2025-12-06T15:37:36.029Z",
  "updatedAt": "2025-12-06T15:37:36.029Z",
  "__v": 0
}
]
```

## 5.Conclusion:

Event-Ease successfully demonstrates a complete full-stack web application built using the MERN stack. Through this project, a functional event registration system was created with real-time data interaction between the frontend and backend. The application supports all major operations including data entry, retrieval, search, update, delete, and export.

This project helped in understanding:

- How frontend and backend communicate through REST APIs
- How MongoDB Atlas stores and manages data
- How React components manage state and re-render efficiently
- How CRUD operations work in real-world applications

Overall, the project fulfills all functional requirements and provides a solid foundation for more advanced features such as admin login, QR code generation, analytics dashboard, and event fee payment integration in the future.